Bhartiya Vidya Bhavans
Sardar Patel Institute of Technology
Munshi Nagar – Andheri West – 400058

Name: Atharv patil (2024300165)
Tanish parakh (2024300157)
Gitesh patil (2024300166)
Branch: C.E

DIVISION: C
BATCH: C1
DATE: 11/08/2026

Experiment No.: 2

Aim : To implement a Java-based Multithreaded Distributed Banking Home Loan Management System demonstrating multithreading and thread synchronization for concurrent loan applications, account EMI transactions, and real-time client notifications from a common server.

Theory :

Multithreading in Distributed Systems is an architecture where a server process spawns multiple execution paths (threads) to handle incoming requests from different clients simultaneously. In modern distributed applications like digital banking portals (SBI, HDFC, ICICI), thousands of user transactions occur concurrently. Without multithreading, a single-threaded server would process client requests sequentially—causing severe latency where one client's execution blocks all subsequent clients.

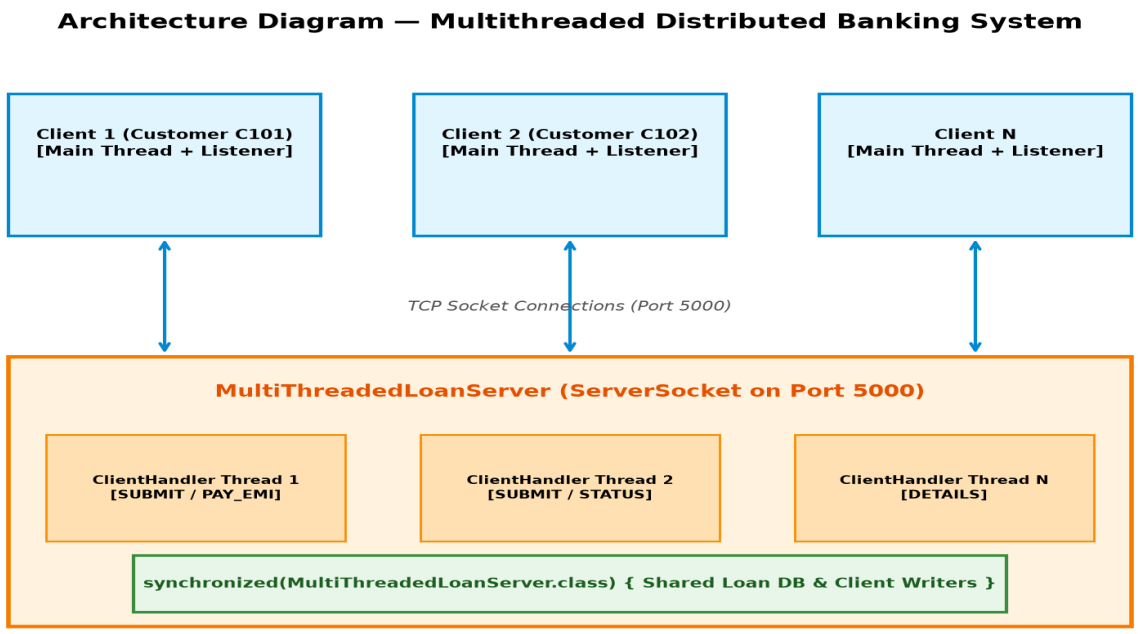
Why Multithreading? Spawning a dedicated ClientHandler thread for each TCP connection (`serverSocket.accept()`) isolates client processing, improves system throughput, and ensures high responsiveness.

Real-Life Examples:
• **Banking Portals:** Multiple account holders pay loan EMIs or submit applications simultaneously without blocking each other.
• **WhatsApp Web & Slack:** Each user sends and receives messages independently via concurrent socket threads.
• **Zoom & Microsoft Teams:** Audio/video streams per participant are processed in separate execution threads.

Thread Synchronization & Concurrency Control: When multiple threads access shared memory (such as loan registries or account balances), race conditions can corrupt banking state. Using Java synchronized blocks ensures mutual exclusion so that critical operations—such as loan ID generation and EMI balance updates—remain thread-safe.

Key Concepts Demonstrated:
● **Client:** Connects to multithreaded server and sends loan requests.
● **Server:** Spawns a dedicated thread per client and manages synchronized banking state.
● **Multithreading:** Enables concurrent, non-blocking execution paths.
● **Concurrency Control:** Protects shared account balances against race conditions.
● **Real-time Broadcast:** Notifies all connected client threads asynchronously of banking transactions.

Problem-solving :



Code :

1. MultiThreadedLoanServer.java (Server with Concurrency Control)

```
import java.io.*;
import java.net.*;
import java.util.*;

public class MultiThreadedLoanServer {

    private static final int PORT = 5000;
    private static Set<PrintWriter> clientWriters = new HashSet<>();
    private static Map<String, LoanRecord> loanDatabase = new HashMap<>();
    private static int loanCounter = 1001;

    // Helper class for stored loan records
    static class LoanRecord {
        String loanId;
        String customerId;
        String customerName;
        double totalAmount;
        double remainingBalance;
        int tenureYears;
        double interestRate;
        String status;

        LoanRecord(String loanId, String customerId, String customerName, double amount, int tenureYears, double
interestRate) {
            this.loanId = loanId;
            this.customerId = customerId;
            this.customerName = customerName;
            this.totalAmount = amount;
            this.remainingBalance = amount;
            this.tenureYears = tenureYears;
            this.interestRate = interestRate;
            this.status = "UNDER_REVIEW";
        }

        @Override
        public String toString() {
            return "Loan ID: " + loanId +
```

```
                " | Customer: " + customerName +
                " (ID: " + customerId + ")" +
                " | Total: ■" + totalAmount +
                " | Balance: ■" + remainingBalance +
                " | Tenure: " + tenureYears + " yrs" +
                " | Interest: " + interestRate + "%" +
                " | Status: " + status;
    }
}

public static void main(String[] args) throws IOException {
    ServerSocket serverSocket = new ServerSocket(PORT);
    System.out.println("================================================");
    System.out.println(" Multithreaded Home Loan Server Started on Port " + PORT);
    System.out.println(" Waiting for client connections...");
    System.out.println("================================================");

    while (true) {
        Socket clientSocket = serverSocket.accept();
        System.out.println("[SERVER LOG] New Client Connected: " + clientSocket.getRemoteSocketAddress());
        new Thread(new ClientHandler(clientSocket)).start();
    }
}

private static class ClientHandler implements Runnable {
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;

    ClientHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);

            synchronized (clientWriters) {
                clientWriters.add(out);
            }

            String request;
            while ((request = in.readLine()) != null) {
                System.out.println("[THREAD " + Thread.currentThread().getId() + "] Request: " + request);
                String response = processRequest(request);
                out.println("RESPONSE|" + response);
            }

        } catch (IOException e) {
            System.out.println("[SERVER LOG] Client disconnected: " + socket.getRemoteSocketAddress());
        } finally {
            synchronized (clientWriters) {
                if (out != null) {
                    clientWriters.remove(out);
                }
            }
            try {
                socket.close();
            } catch (IOException e) {
                // Ignore close exception
            }
        }
    }

    private String processRequest(String request) {
        try {
            String[] data = request.split("\\|");
            String command = data[0];
```

```
                if (command.equals("SUBMIT")) {
                    String customerId = data[1];
                    String customerName = data[2];
                    double amount = Double.parseDouble(data[3]);
                    int tenure = Integer.parseInt(data[4]);
                    double interest = Double.parseDouble(data[5]);

                    String loanId;
                    LoanRecord record;

                    // CONCURRENCY CONTROL: Synchronize ID generation & DB insertion
                    synchronized (MultiThreadedLoanServer.class) {
                        loanId = "HL" + loanCounter++;
                        record = new LoanRecord(loanId, customerId, customerName, amount, tenure, interest);
                        loanDatabase.put(loanId, record);
                    }

                    // Broadcast notification to all active clients
                    broadcast("[NOTIFICATION] New Loan Application Registered: " + loanId + " by " + customerName
);

                    return "Loan Application Submitted Successfully! " + record.toString();
                }
                else if (command.equals("PAY_EMI")) {
                    String loanId = data[1];
                    double paymentAmount = Double.parseDouble(data[2]);

                    // CONCURRENCY CONTROL: Synchronize transaction processing per loan ID
                    synchronized (MultiThreadedLoanServer.class) {
                        if (!loanDatabase.containsKey(loanId)) {
                            return "ERROR: Loan ID " + loanId + " not found.";
                        }
                        LoanRecord record = loanDatabase.get(loanId);
                        if (record.remainingBalance <= 0) {
                            return "Loan ID " + loanId + " is already FULLY_PAID!";
                        }

                        record.remainingBalance -= paymentAmount;
                        if (record.remainingBalance <= 0) {
                            record.remainingBalance = 0;
                            record.status = "CLOSED_FULLY_PAID";
                        } else {
                            record.status = "ACTIVE_APPROVED";
                        }

                        broadcast("[NOTIFICATION] Payment of ■" + paymentAmount + " processed for " + loanId + ".
 Remaining: ■" + record.remainingBalance);
                        return "Payment Received! Updated Record: " + record.toString();
                    }
                }
                else if (command.equals("STATUS")) {
                    String loanId = data[1];
                    synchronized (MultiThreadedLoanServer.class) {
                        if (loanDatabase.containsKey(loanId)) {
                            LoanRecord record = loanDatabase.get(loanId);
                            return "Loan ID: " + record.loanId + " | Status: " + record.status + " | Balance: ■"
+ record.remainingBalance;
                        } else {
                            return "ERROR: Loan ID " + loanId + " not found.";
                        }
                    }
                }
                else if (command.equals("DETAILS")) {
                    String loanId = data[1];
                    synchronized (MultiThreadedLoanServer.class) {
                        if (loanDatabase.containsKey(loanId)) {
                            return loanDatabase.get(loanId).toString();
                        } else {
                            return "ERROR: Loan ID " + loanId + " not found.";
```

```
                    }
                }
            }
            else {
                return "ERROR: Invalid command.";
            }

        } catch (Exception e) {
            return "ERROR processing request: " + e.getMessage();
        }
    }

    private void broadcast(String message) {
        synchronized (clientWriters) {
            for (PrintWriter writer : clientWriters) {
                writer.println("BROADCAST|" + message);
            }
        }
    }
}
```

2. LoanClient.java (Multithreaded Client with Listener Thread)

```
import java.io.*;
import java.net.*;
import java.util.*;

public class LoanClient {

    private static final String SERVER_HOST = "localhost";
    private static final int SERVER_PORT = 5000;

    public static void main(String[] args) {
        try {
            Socket socket = new Socket(SERVER_HOST, SERVER_PORT);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in);

            System.out.println("==================================================");
            System.out.println(" MULTITHREADED DISTRIBUTED HOME LOAN CLIENT");
            System.out.println("==================================================");

            // MULTITHREADING: Background listener thread for server responses and real-time broadcasts
            Thread listenerThread = new Thread(() -> {
                try {
                    String serverMessage;
                    while ((serverMessage = in.readLine()) != null) {
                        if (serverMessage.startsWith("BROADCAST|")) {
                            System.out.println("\n[REAL-TIME BROADCAST] " + serverMessage.substring(10));
                            System.out.print("Enter choice: ");
                        } else if (serverMessage.startsWith("RESPONSE|")) {
                            System.out.println("\n[SERVER RESPONSE] " + serverMessage.substring(9));
                        } else {
                            System.out.println("\n[SERVER MESSAGE] " + serverMessage);
                        }
                    }
                } catch (IOException e) {
                    System.out.println("\n[DISCONNECTED] Server connection closed.");
                }
            });
            listenerThread.setDaemon(true);
            listenerThread.start();

            // Main thread: Handles CLI menu and sends requests
            while (true) {
                printMenu();
                String choice = scanner.nextLine().trim();

                if (choice.equals("1")) {
```

```
                    System.out.print("Customer ID: ");
                    String customerId = scanner.nextLine().trim();

                    System.out.print("Customer Name: ");
                    String customerName = scanner.nextLine().trim();

                    System.out.print("Loan Amount (■): ");
                    double amount = Double.parseDouble(scanner.nextLine().trim());

                    System.out.print("Loan Tenure (years): ");
                    int tenure = Integer.parseInt(scanner.nextLine().trim());

                    System.out.print("Interest Rate (%): ");
                    double interest = Double.parseDouble(scanner.nextLine().trim());

                    String request = "SUBMIT|" + customerId + "|" + customerName + "|" + amount + "|" + tenure +
"|" + interest;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("2")) {
                    System.out.print("Enter Loan ID: ");
                    String loanId = scanner.nextLine().trim();

                    System.out.print("Enter Payment / EMI Amount (■): ");
                    double amount = Double.parseDouble(scanner.nextLine().trim());

                    String request = "PAY_EMI|" + loanId + "|" + amount;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("3")) {
                    System.out.print("Enter Loan ID: ");
                    String loanId = scanner.nextLine().trim();


                    String request = "STATUS|" + loanId;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("4")) {
                    System.out.print("Enter Loan ID: ");
                    String loanId = scanner.nextLine().trim();

                    String request = "DETAILS|" + loanId;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("5")) {
                    System.out.println("Exiting Distributed Home Loan Client...");
                    socket.close();
                    break;
                } else {
                    System.out.println("Invalid choice. Please enter 1-5.");
                }
            }

            scanner.close();

        } catch (Exception e) {
            System.err.println("Error connecting to server: " + e.getMessage());
        }
    }

    private static void printMenu() {
        System.out.println("\n-----------------------------------------------");
        System.out.println("1. Apply for Home Loan (SUBMIT)");
        System.out.println("2. Make EMI / Account Payment (PAY_EMI)");
        System.out.println("3. Check Loan Status (STATUS)");
        System.out.println("4. View Loan Details (DETAILS)");
        System.out.println("5. Exit");
        System.out.print("Enter choice: ");
```

```
    }
}
```

Output screenshot :

Figure 1: Multithreaded Server Started and Listening on Port 5000

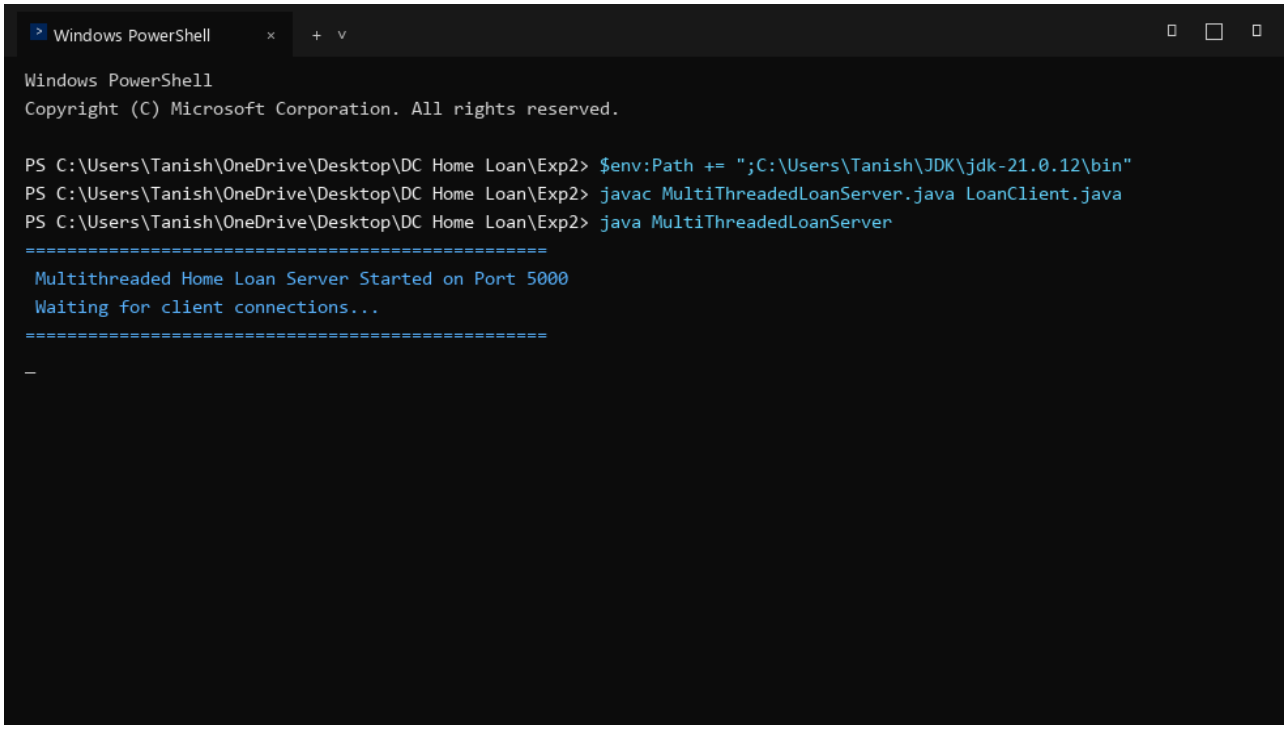


Figure 2: Client 1 Submitting Home Loan Application (HL1001)

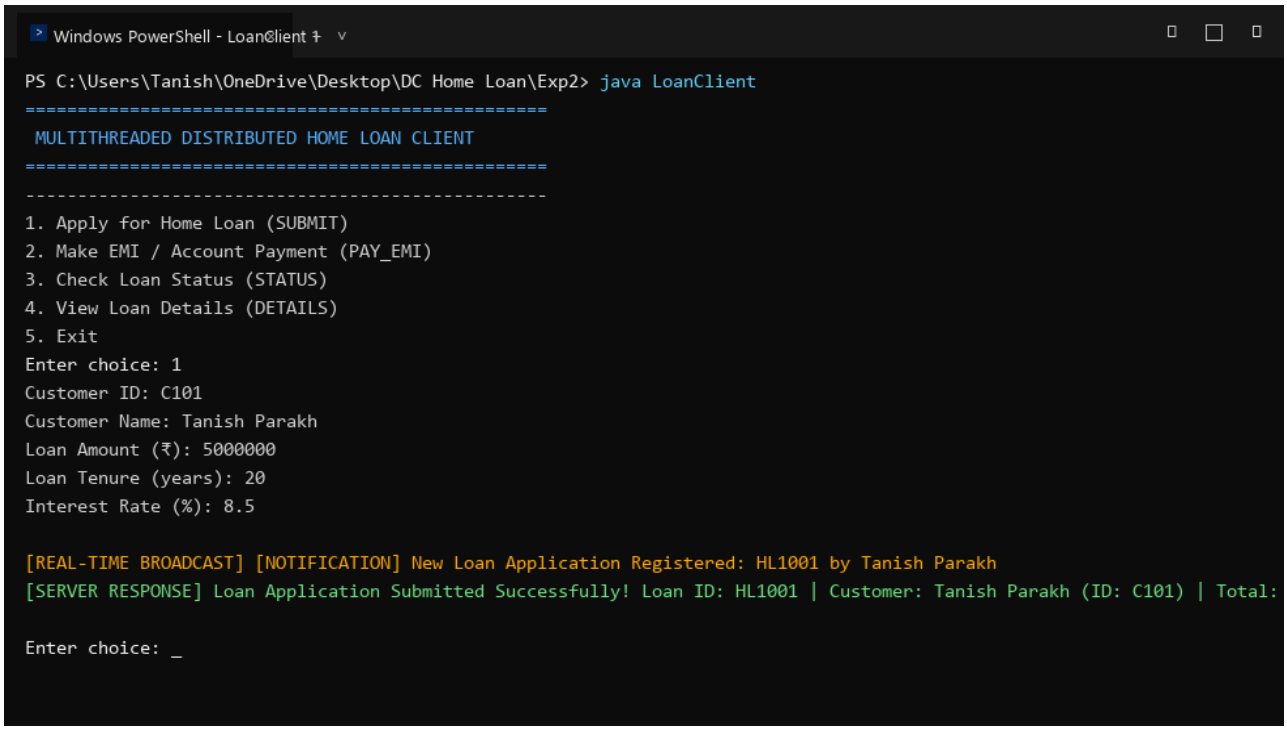


Figure 3: Client 1 Executing Synchronized EMI Payment Transaction (Balance updated)

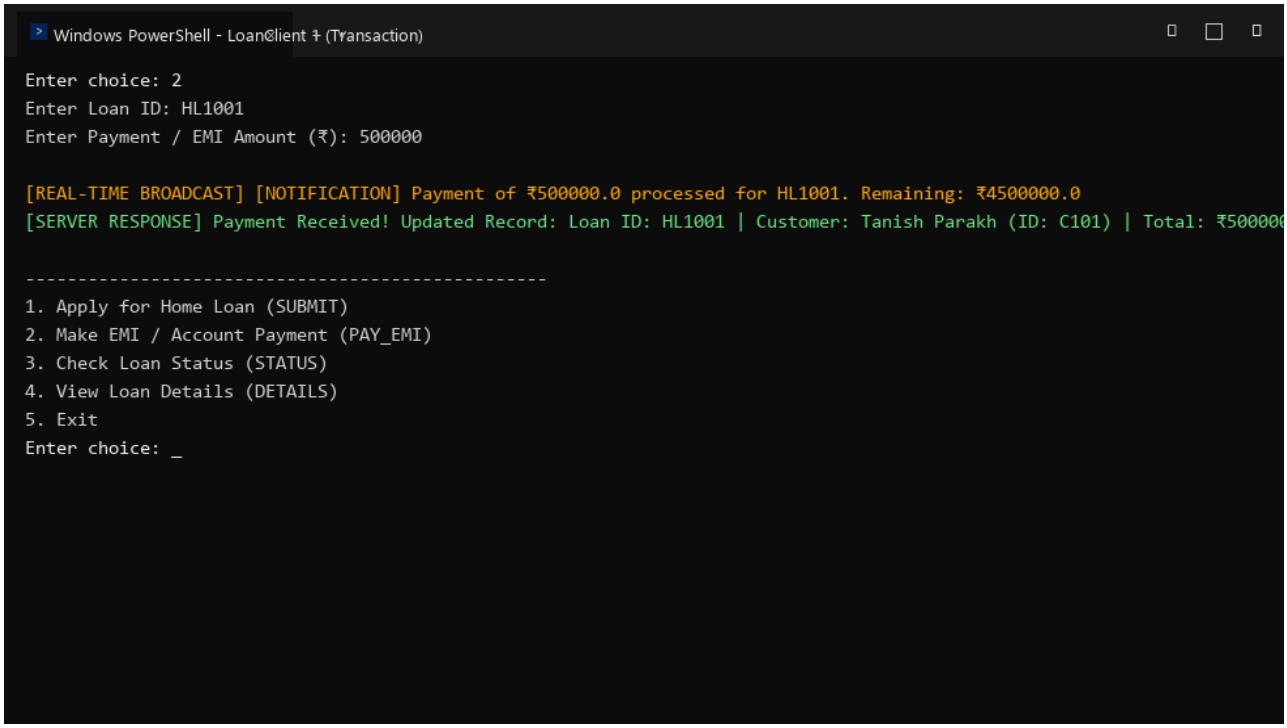


Figure 4: Client 2 Receiving Real-Time Async Broadcast Notification

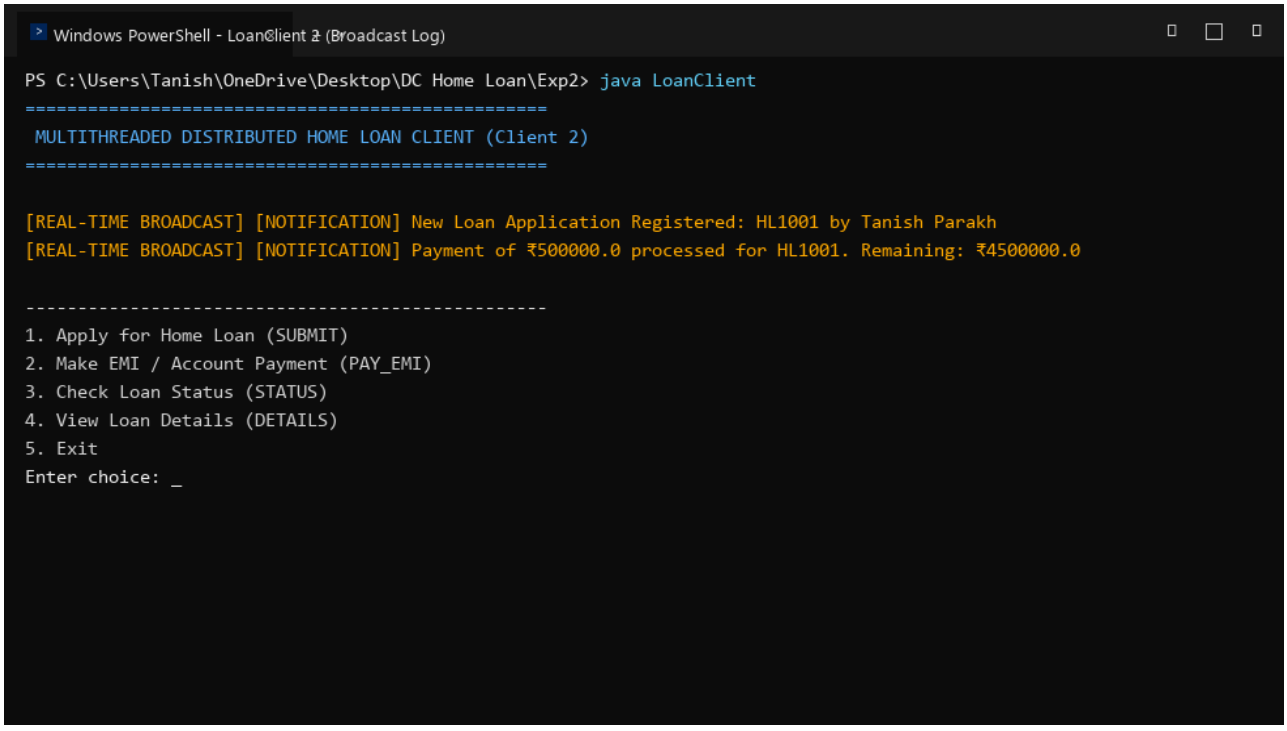
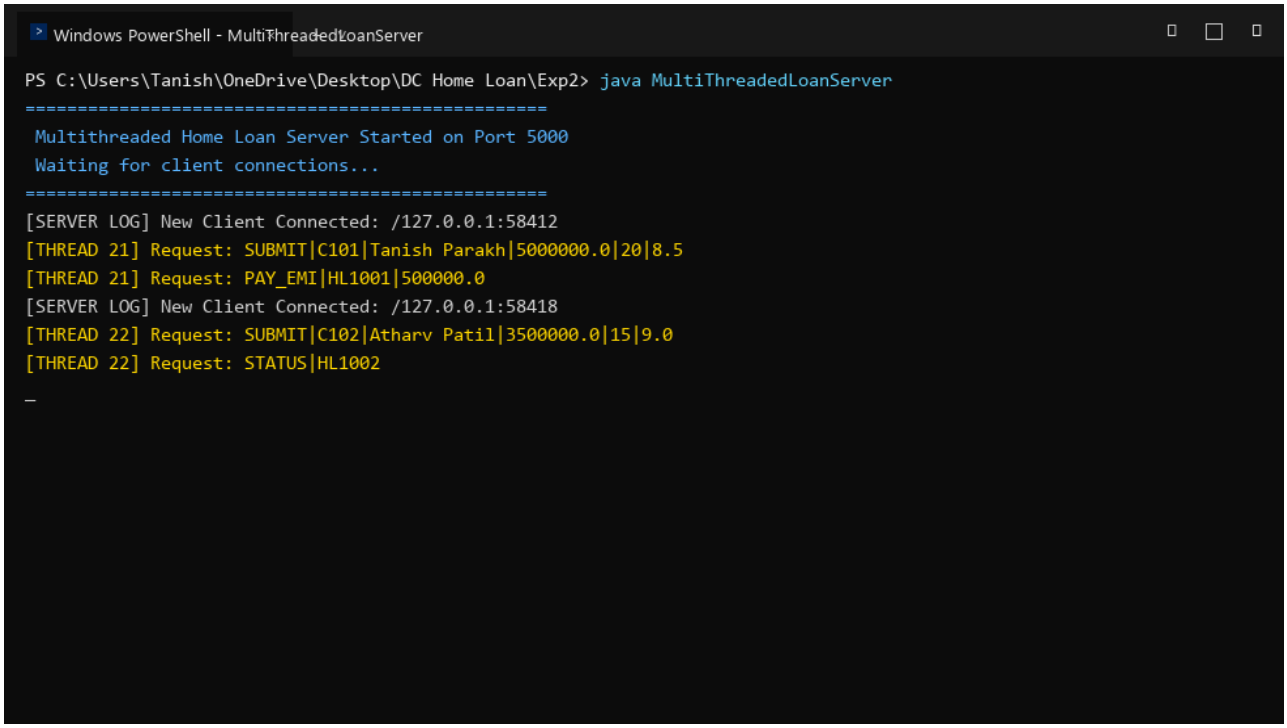


Figure 5: Multithreaded Server Execution Log Showing Dedicated Client Threads ([THREAD id])



Conclusion: (It should include what the student has learned/ understood from the experiment).

In this experiment, we successfully implemented a multithreaded distributed banking home loan management system in Java. We demonstrated how spawning dedicated ClientHandler threads for incoming client socket connections enables high parallelism and non-blocking operation. Furthermore, we implemented strict concurrency control using Java synchronized blocks to safeguard shared account databases and loan records against race conditions. The experiment effectively highlighted the critical role of multithreading, thread safety, and real-time asynchronous broadcasting in modern distributed banking software.

Attach research paper based on experiment performed into the classroom (Not for all the practicals)

(Students are required to prepare one doc file for each experiment and then upload it in pdf form.)